

PROJECT REQUIREMENTS DOCUMENT

Temporal Blood Biomarker Analysis for Kidney Disease Detection

Experiential Learning (EL) Main Project

RV College of Engineering | Department of CS-AIML

Document Type: Product Requirements Document (PRD)

Project Phase Coverage: Phase 1, Phase 2, Phase 3

Primary Dataset: MIMIC-IV Demo v2.2 (physionet.org — Free, Open Access)

Models: XGBoost (tabular) + LSTM (temporal sequences)

Version: 1.0 | March 2026

1. Project Overview

1.1 Project Identity

Field	Details
Project Title	Temporal Blood Biomarker Analysis for Kidney Disease Detection
Short Name	TBBAKD
SDG Alignment	SDG 3 — Good Health and Well-being
Domain	Healthcare / Clinical Decision Support / Predictive Medicine
Dataset	MIMIC-IV Clinical Database Demo v2.2
Dataset URL	https://physionet.org/content/mimic-iv-demo/2.2/
Access	Free PhysioNet account (no CITI course, no institution required)
ML Models	XGBoost (admission-level) + LSTM (multi-visit sequences)
Explainability	SHAP TreeExplainer — global summary + per-patient waterfall plots
Application	Streamlit web app — accepts multi-visit blood values, outputs risk score + SHAP explanation
Timeline	20 weeks across 3 phases
Target Venue	IEEE Access / MDPI Applied Sciences / IEEE CBMS Conference

1.2 Problem Statement

Chronic Kidney Disease (CKD) is a silent, progressive condition where the kidneys gradually lose their ability to filter waste from the blood. It affects roughly 1 in 10 people worldwide and is responsible for over 1.2 million deaths annually. The most dangerous characteristic of CKD is that it produces no noticeable symptoms until 70 to 80 percent of kidney function is already lost — by which point the disease is difficult and expensive to manage.

Blood tests ordered routinely by doctors — measuring creatinine, blood urea nitrogen (BUN), potassium, sodium, and hemoglobin — contain early warning signals of kidney deterioration. However, virtually all published machine learning systems treat each blood test as a single isolated snapshot, ignoring the trend of how these values change across multiple visits over time.

The core insight this project is built on: a patient whose creatinine has risen from 1.1 to 1.4 to 1.8 mg/dL across three months is at dramatically greater risk than a patient who has been stable at 1.8 mg/dL for years. Both patients look identical to a single-snapshot classifier. Only a system that reads the trajectory can tell the difference. This project builds exactly that system — treating a patient's blood test history as a time-ordered sequence, learning to detect deterioration trends, and issuing early warnings before the disease reaches a critical stage.

1.3 Objectives

1. Build a multi-visit blood report analysis pipeline that models kidney health as a time-series, not a single snapshot.
2. Engineer clinically meaningful temporal features — especially the creatinine slope across visits — that capture how fast kidney function is changing.
3. Train two complementary ML models: XGBoost for per-admission risk scoring with engineered temporal features, and LSTM for sequential visit-level prediction directly from the raw time-series.
4. Generate SHAP-based explanations that tell a doctor which blood values — and which trends — drove the risk score for each individual patient.
5. Build a functional Streamlit web application where a doctor enters blood values from multiple visits and receives a time-aware CKD risk assessment with explanations.
6. Produce a research paper documenting the methodology, experiments, and results suitable for submission to an indexed journal or IEEE conference.

1.4 Novelty — How This Differs from Existing Work

The table below compares this project to the dominant pattern in the CKD ML literature (UCI CKD dataset, single-snapshot, standard classifiers):

Dimension	What Existing Papers Do	What This Project Does
Data structure	Single blood test per patient — one row, no timestamps	Sequence of multiple visits per patient — chronologically ordered
Primary dataset	UCI CKD — 400 patients, pre-cleaned, no visit dates	MIMIC-IV Demo — real hospital data with timestamps, noise, irregular gaps
Key novel feature	Raw blood values at one point in time	Creatinine slope across visits — rate of change as an explicit feature
Ground-truth labels	Pre-labelled UCI column or arbitrary thresholds	ICD-10 discharge diagnosis codes (N18.1-N18.9) from real doctors
Explainability	SHAP on static features only ('albumin was high')	SHAP on temporal features — explains trajectory, not just current state
Model architecture	Single classifier: Random Forest, XGBoost, SVM, or KNN	Dual architecture: XGBoost (engineered features) + LSTM (raw sequence)
Clinical question	Is CKD present right now?	Is kidney function deteriorating over time?

2. Dataset Specification

2.1 MIMIC-IV Demo v2.2 — Overview

The Medical Information Mart for Intensive Care IV (MIMIC-IV) is a large, freely available database of de-identified health data from patients admitted to Beth Israel Deaconess Medical Center (BIDMC), Boston, between 2008 and 2019. The demo version contains a representative subset of approximately 100 patients and is openly accessible under the Open Data Commons Open Database License v1.0, without requiring institutional credentials or a CITI ethics course.

Property	Value
Full name	MIMIC-IV Clinical Database Demo
Version	2.2
Direct URL	https://physionet.org/content/mimic-iv-demo/2.2/
License	Open Data Commons Open Database License (ODbL) v1.0
Login required	Free PhysioNet account only — email verification, no waiting period
Patients	~100 de-identified ICU and hospital patients
Lab events	~107,000 rows across ~500+ distinct lab test types
Key folder	hosp/ (hospital module — contains all lab and diagnosis data)
Files used	labevents.csv, diagnoses_icd.csv, d_labitems.csv, admissions.csv

2.2 Files and Their Role in This Project

File	Contains	How It Is Used
labevents.csv	One row per lab test result: patient ID, admission ID, timestamp, item ID, numeric result, normal/abnormal flag, reference range upper and lower bounds	Primary data source. Filtered to 8 kidney tests, aggregated per admission, pivoted to wide format. Forms the XGBoost training table and the raw material for LSTM sequences.
diagnoses_icd.csv	ICD-10 diagnosis codes assigned by doctors at patient discharge, one row per diagnosis per admission	Used to derive ground-truth CKD labels. Patients with ICD codes N18.1 through N18.9 are labelled ckd_label = 1.
d_labitems.csv	Dictionary table mapping numeric item IDs to human-readable lab test names, fluid type, and category	Used to identify the exact item IDs corresponding to the 8 kidney-relevant tests.
admissions.csv	One row per hospital admission: patient ID, admission datetime,	Used to get the start timestamp of each admission for sorting visits chronologically per patient.

File	Contains	How It Is Used
	discharge datetime, admission type	

2.3 The 8 Blood Parameters Used

Only these 8 lab tests are used as model features. All other tests in labevents.csv are discarded.

Test Name	MIMIC Item ID	Normal Range	Clinical Relevance to Kidneys
Creatinine	50912	0.6 – 1.2 mg/dL	Primary waste marker produced by muscles and filtered by kidneys. Rising creatinine directly indicates declining kidney filtration efficiency.
BUN	51006	7 – 20 mg/dL	Blood Urea Nitrogen — second waste marker from protein breakdown. Rises when kidneys cannot clear urea. Elevated BUN + Creatinine = strong CKD signal.
Potassium	50971	3.5 – 5.0 mEq/L	Electrolyte tightly regulated by healthy kidneys. High potassium (hyperkalemia) in later CKD stages is dangerous for heart rhythm.
Sodium	50983	136 – 145 mEq/L	Water-salt balance controlled by kidneys. Imbalance indicates kidney regulation is failing.
Hemoglobin	51222	12 – 17 g/dL	Kidneys produce EPO, the hormone that signals red blood cell production. Low hemoglobin can indicate kidney EPO signaling is declining.
Bicarbonate	50882	22 – 29 mEq/L	Acid-base buffer. Kidneys regulate acid levels. Low bicarbonate (metabolic acidosis) is a common consequence of CKD.
Calcium	50893	8.5 – 10.5 mg/dL	Mineral regulated in part by kidneys. Imbalances appear in advancing CKD when the kidney-bone-mineral axis breaks down.
Glucose	50931	70 – 100 mg/dL	Diabetes is the single largest cause of CKD globally. Chronically elevated glucose damages kidney blood vessels over years.

2.4 Labelling Strategy

Primary Method — ICD-10 Diagnosis Codes

CKD ground-truth labels are derived from diagnoses_icd.csv using the following ICD-10-CM codes representing CKD Stages 1 through 5 and End Stage Renal Disease:

ICD-10 Code	Meaning
N18.1	CKD Stage 1 — kidney damage with normal or high eGFR (> 90)
N18.2	CKD Stage 2 — mild reduction in eGFR (60–89)
N18.30 / N18.31 / N18.32	CKD Stage 3 — moderate reduction in eGFR (30–59)
N18.4	CKD Stage 4 — severe reduction in eGFR (15–29)
N18.5	CKD Stage 5 — kidney failure, eGFR < 15
N18.6	End Stage Renal Disease — dialysis dependent
N18.9	CKD, unspecified stage

A patient-level binary label `ckd_label = 1` is assigned if ANY of the above ICD codes appear in any of that patient's admissions. This label is merged into the feature table on `subject_id`.

Fallback Method — KDIGO Clinical Rule

If the ICD file is unavailable or yields too few positive labels, a clinically validated rule based on the KDIGO (Kidney Disease: Improving Global Outcomes) definition is applied. A patient is labelled CKD = 1 if all three of the following are true: (1) mean creatinine across all admissions exceeds 1.3 mg/dL, (2) creatinine was flagged as abnormal in two or more separate hospital admissions, and (3) at least 90 days elapsed between the first and last abnormal creatinine reading. This operationalises the KDIGO definition of chronicity — kidney damage present for 3 or more months.

3. System Architecture

3.1 End-to-End Pipeline

Stage	Input	Process	Output
1. Data Ingestion	labevents.csv, diagnoses_icd.csv, d_labitems.csv, admissions.csv	Filter to 8 kidney tests, merge with diagnosis labels, sort chronologically per patient	Cleaned, labelled, time-ordered lab event table
2. Feature Engineering	Cleaned lab event table	Aggregate per admission (mean, max, ever_abnormal), compute creatinine slope across admissions, pivot to wide format, build LSTM sequences	admission_features.csv (XGBoost input) and X_sequences.npy / y_labels.npy (LSTM input)
3a. XGBoost Training	admission_features.csv	SMOTE balancing, MinMaxScaler, XGBoost training, 5-fold CV, SHAP explanation generation	ckd_model_xgb.pkl, scaler.pkl, feature_cols.pkl, SHAP plots
3b. LSTM Training	X_sequences.npy, y_labels.npy	Sequence scaling, LSTM architecture, training with EarlyStopping, evaluation	ckd_model_lstm.keras, seq_scaler.pkl, training loss plots
4. Comparison	Both model evaluation results	Side-by-side table: XGBoost vs LSTM on AUC, F1, recall. Ablation: with slope vs without slope.	Model comparison table — key result for research paper
5. Application Layer	Doctor-entered multi-visit values	Prepare features from user input, run both models, generate SHAP waterfall, optionally call LLM	Risk score (%), creatinine trend indicator, SHAP plot, plain-English summary

3.2 Feature Engineering — Full Specification

Per-Admission Features (XGBoost input — 25 features total)

For each hospital admission, the following statistics are computed across all lab readings of each of the 8 tests within that admission:

Feature Pattern	Formula	Example	What It Captures
mean_[TestName]	Mean of all readings of that test during the admission	mean_Creatinine = 1.6	Typical level during this hospital stay
max_[TestName]	Maximum reading of that test during the admission	max_Creatinine = 2.1	Peak / worst-case level —

Feature Pattern	Formula	Example	What It Captures
			highest stress observed
ever_abnormal_[TestName]	1 if the flag column was 'abnormal' at least once, else 0	ever_abnormal_BUN = 1	Whether this test was clinically flagged during the stay
creat_slope	Slope of linear regression fitted to mean_Creatinine values across admissions, per patient	creat_slope = +0.35	Rate of creatinine rise — the primary temporal novelty feature

Feature count: 8 tests x 3 statistics (mean, max, ever_abnormal) = 24 features, plus creat_slope = 25 features total per admission row.

LSTM Sequence Format

Each patient is represented as a 3-dimensional array of shape (N_visits, 8), where N_visits is fixed at 3 (the number of consecutive admissions used as input). Each position in the sequence is a visit snapshot: the mean value of each of the 8 tests during that admission. Admissions are sorted chronologically — oldest first, most recent last. The full training array has shape (N_patients, 3, 8). Patients with fewer than 4 admissions (3 input + 1 outcome) are excluded.

3.3 Model Architectures

Model A — XGBoost

Parameter	Value and Justification
Library	xgboost — XGBClassifier
n_estimators	200 — sufficient for 25 features; more trees improve stability on small datasets
max_depth	4 — shallow trees prevent overfitting on approximately 251 training admissions
learning_rate	0.1 — standard starting point; reduce to 0.05 if overfitting observed
subsample	0.8 — use 80% of training data per tree for regularisation
eval_metric	logloss — appropriate for binary classification probability outputs
Imbalance handling	SMOTE (imbalanced-learn) — synthesises minority class samples before training
Scaling	MinMaxScaler — fitted on X_train only; applied to both X_train and X_test
Validation	5-fold stratified cross-validation + 80/20 held-out test set

Parameter	Value and Justification
Explainability	shap.TreeExplainer — global summary_plot + per-patient waterfall_plot
Saved files	ckd_model_xgb.pkl, scaler.pkl, feature_cols.pkl (via joblib)

Model B — LSTM

Parameter	Value and Justification
Framework	TensorFlow / Keras
Input shape	(3, 8) — 3 consecutive visits, 8 blood test features per visit
Layer 1	LSTM(64 units, return_sequences=False) — reads the 3-visit sequence
Layer 2	Dropout(0.2) — prevents overfitting on small training set
Layer 3	Dense(32, activation='relu') — learns non-linear combinations of LSTM output
Output layer	Dense(1, activation='sigmoid') — outputs probability between 0 and 1
Optimizer	Adam with learning rate 0.001 (default)
Loss	binary_crossentropy — standard for binary classification
Epochs	50 maximum, with EarlyStopping(patience=10, monitor='val_loss')
Batch size	16 — small batches appropriate for approximately 80 training sequences
Scaling	MinMaxScaler applied to each feature independently across all visits and patients
Saved files	ckd_model_lstm.keras (or .h5), seq_scaler.pkl

3.4 Evaluation Metrics

Standard accuracy is NOT reported as the primary metric due to potential class imbalance. All of the following are computed for both models:

Metric	Target Value	Why It Matters
AUC-ROC	> 0.85	Primary metric. Measures overall discriminative ability. Not affected by class imbalance.
Recall (Sensitivity)	> 0.85	Of all patients who actually have CKD, what fraction did the model correctly identify? Missing a sick patient is the worst error in medical AI.
Precision	> 0.75	Of all patients the model flagged as CKD, what fraction actually have it? Controls false alarm rate.
F1 Score	> 0.80	Harmonic mean of precision and recall. Primary balance metric.

Metric	Target Value	Why It Matters
5-fold CV AUC mean $\pm$ std	Reported	More reliable than a single train/test split on a small dataset. Shows how stable the model is across different data splits.
Confusion Matrix	Visualised	Shows true positives, true negatives, false positives, false negatives in a 2x2 grid.

4. Technology Stack

4.1 Tools and Platforms

Tool	Purpose	Cost	URL
Google Colab	Primary coding and model training environment. Free GPU access.	Free	colab.research.google.com
Google Drive	Store datasets, trained .pkl files, notebooks across sessions	Free (15 GB)	drive.google.com
GitHub	Version control and source for Streamlit Community Cloud deployment	Free	github.com
Streamlit	Web application framework — no HTML/CSS knowledge required	Free	streamlit.io
Streamlit Cloud	Free app hosting — connects to GitHub repo, generates public URL	Free	share.streamlit.io
PhysioNet	Source for MIMIC-IV Demo dataset download	Free account	physionet.org
draw.io	Architecture diagram creation for Phase 1 PPT	Free	app.diagrams.net

4.2 Python Libraries

Library	Version	Purpose
pandas	latest	Data loading, cleaning, filtering, groupby aggregation, pivoting, merging
numpy	latest	Array operations, sequence construction for LSTM, slope computation with polyfit
scikit-learn	latest	MinMaxScaler, train_test_split, cross_val_score, classification_report, roc_auc_score
xgboost	latest	XGBClassifier — primary tabular prediction model
imbalanced-learn	latest	SMOTE — oversamples minority class before XGBoost training
tensorflow	2.x	LSTM model definition, training, EarlyStopping callback, model saving and loading
shap	latest	TreeExplainer for XGBoost — global summary_plot and per-patient waterfall_plot
matplotlib	latest	Confusion matrix heatmap, SHAP plot rendering, LSTM loss curve plotting
seaborn	latest	Styled confusion matrix visualisation

Library	Version	Purpose
joblib	latest	Save and load .pkl files for model and scaler
streamlit	latest	Full web application UI — runs with 'streamlit run app.py'
pymupdf	latest	Extract text from computer-generated blood report PDFs
pytesseract	latest	OCR fallback for scanned / image-based blood report PDFs

Install all at once: `pip install pandas numpy scikit-learn xgboost imbalanced-learn tensorflow shap matplotlib seaborn joblib streamlit pymupdf pytesseract`

5. Functional Requirements

5.1 Data Pipeline

ID	Requirement	Acceptance Criterion
DP-01	Load labevents.csv and filter rows to the 8 kidney-relevant item IDs	Filtered dataframe contains exactly 8 unique test_name values
DP-02	Drop rows where valuenum is null (non-numeric lab results)	No NaN values in valuenum column after filtering
DP-03	Map item IDs to human-readable names using the kidney_item_ids dictionary	All 8 item IDs mapped correctly; 'test_name' column present
DP-04	Load diagnoses_icd.csv and identify rows with ICD codes N18.1 through N18.9	patient_ckd DataFrame with subject_id and ckd_label columns created
DP-05	Merge lab features with CKD labels on subject_id	No patients lost; ckd_label column present in final table
DP-06	Sort all lab events by charttime within each subject_id	Events per patient are in ascending chronological order
DP-07	Aggregate per admission: compute mean_val, max_val, ever_abnormal for each of the 8 tests	Output has 24 aggregated feature columns per admission row (8 tests x 3 stats)
DP-08	Compute creatinine slope per patient across admissions using np.polyfit	creat_slope column present; positive for patients with rising creatinine across visits
DP-09	Pivot aggregated table to wide format with one row per admission	Shape: approximately 251 rows x 26 columns (25 features + 1 label)
DP-10	Build LSTM sequence array of shape (N_patients, 3, 8)	X_sequences.npy loadable with np.load; confirmed shape
DP-11	Fill remaining NaN values in feature table with column median	feature_table.isnull().sum().sum() == 0 after fill

5.2 XGBoost Model

ID	Requirement	Acceptance Criterion
XGB-01	Apply SMOTE to balance class distribution before training	value_counts() of both classes are equal after SMOTE
XGB-02	Fit MinMaxScaler on X_train only; transform both X_train and X_test with the same fitted scaler	All values are between 0 and 1 after scaling; scaler was NOT fit on X_test
XGB-03	Train XGBoost with specified hyperparameters	model.fit() completes without errors; training loss decreases over iterations
XGB-04	Evaluate on held-out 20% test set; report classification_report and roc_auc_score	AUC >= 0.80 on test set (target: > 0.85)

ID	Requirement	Acceptance Criterion
XGB-05	Run 5-fold stratified cross-validation; report mean AUC and standard deviation	Output printed: 'Mean AUC: X.XXX +/- X.XXX'
XGB-06	Generate and save SHAP global summary plot	shap_summary.png saved and interpretable
XGB-07	Generate SHAP waterfall plot for at least one individual patient in the test set	Waterfall plot shows feature name, SHAP value, and direction for top 10 features
XGB-08	Save ckd_model_xgb.pkl and scaler.pkl using joblib.dump()	joblib.load() successfully reloads both without errors
XGB-09	Save feature column list as feature_cols.pkl	Loaded list matches training columns exactly in same order

5.3 LSTM Model

ID	Requirement	Acceptance Criterion
LSTM-01	Confirm input array shape is (N_patients, 3, 8) before training	X_sequences.npy shape printed and confirmed
LSTM-02	Scale sequence data using MinMaxScaler applied across all patients and visits per feature	All values in the sequence array are between 0 and 1
LSTM-03	Build LSTM network: LSTM(64) -> Dropout(0.2) -> Dense(32, relu) -> Dense(1, sigmoid)	model.summary() output matches this architecture
LSTM-04	Train with EarlyStopping(patience=10, monitor='val_loss')	Training stops automatically; does not run all 50 epochs if val_loss stops improving
LSTM-05	Evaluate using same metrics as XGBoost: AUC, F1, precision, recall	Metrics table produced for direct comparison with XGBoost results
LSTM-06	Plot training loss and validation loss curves across epochs	Plot saved as lstm_loss_curves.png; no severe divergence between train and val curves
LSTM-07	Save trained model as ckd_model_lstm.keras	tf.keras.models.load_model() reloads model and makes predictions without errors

5.4 Web Application

ID	Requirement	Acceptance Criterion
APP-01	Load ckd_model_xgb.pkl, scaler.pkl, feature_cols.pkl on startup using st.cache_resource	App starts without errors; loading occurs once and is cached
APP-02	User selects number of past blood report visits via slider (range: 2 to 5)	UI renders correct number of visit input blocks based on slider value

ID	Requirement	Acceptance Criterion
APP-03	User enters 8 blood values for each visit using number_input fields with correct min/max and unit labels	All 8 fields present per visit block; units and normal ranges shown as help text
APP-04	On clicking Analyse, app correctly computes mean, max, ever_abnormal, and creat_slope from entered values	No KeyError, no shape mismatch, no NaN in final input vector
APP-05	App displays risk score as percentage with colour coding: green below 35%, amber 35-65%, red above 65%	Correct colour shown for test values in each band
APP-06	App displays creatinine trend direction with plain-English caption: rising / stable / falling	Trend correctly reflects computed slope; arrow or indicator present
APP-07	App renders SHAP waterfall plot for the current patient inline using st.pyplot()	Plot visible in app; top 10 features shown with SHAP values and directions
APP-08	App optionally loads LSTM model and shows both XGBoost and LSTM risk scores side by side	Two score cards rendered; labelled as XGBoost and LSTM respectively
APP-09	App displays research disclaimer at top of page	Disclaimer text visible: 'For research purposes only. Not a substitute for medical advice.'

6. Phase-Wise Execution Plan

Phase 1 — Research and Proposal (Weeks 1–3)

Deliverable: PowerPoint Presentation submitted to faculty

Week	Task	Output
Week 1	Read CKD background (Wikipedia + 1 YouTube video on CKD stages). Search Google Scholar for 5 papers on 'CKD machine learning blood test'. Read only the abstract of each. Note what each paper did and its reported accuracy.	List of 5 papers with one-line summaries for literature review slides
Week 1	Create free PhysioNet account at physionet.org . Download MIMIC-IV Demo from physionet.org/content/mimic-iv-demo/2.2/ . Unzip archive. Confirm hosp/ folder and its CSV files are accessible.	Local copy of <code>labevents.csv</code> and <code>diagnoses_icd.csv</code> confirmed on your machine
Week 2	Open <code>labevents.csv</code> in Google Colab. Run basic exploration: <code>df.shape</code> , <code>df['itemid'].nunique()</code> , <code>df['flag'].value_counts()</code> , <code>df.isnull().sum()</code> . Identify the 8 kidney-relevant item IDs using <code>d_labitems.csv</code> .	Exploration notebook with a clear understanding of the file structure and 8 target item IDs identified
Week 2	Draw the system architecture pipeline diagram: Multi-Visit Input -> Feature Engineering (slope, mean, max) -> XGBoost + LSTM -> SHAP -> Streamlit App. Use draw.io (free at app.diagrams.net) or PowerPoint SmartArt.	Architecture diagram image ready for PPT inclusion
Week 3	Build the 10-slide Phase 1 PPT. Use Google Slides with the Simple Light theme. Include all sections listed below.	Phase 1 PPT completed and submitted

Phase 1 PPT — Required Slides

- Title Slide — project name, student name, college, department, date
- Problem Statement — CKD global statistics, why early detection fails, the snapshot vs trend gap with a simple analogy (photo vs movie)
- Objectives — 6 objectives from Section 1.3
- SDG Alignment — SDG 3 logo (download from sdgs.un.org), 2-sentence explanation of why CKD detection aligns with SDG 3
- Literature Review — 5 papers: each paper gets one row in a table showing: what they did, their accuracy, and their key limitation (all used single-snapshot data, not time-series)
- Dataset — MIMIC-IV Demo: size, URL, what `labevents.csv` contains, how ICD-10 codes from `diagnoses_icd.csv` are used for labels
- Proposed Methodology — the pipeline diagram from Week 2, with a 3-sentence explanation of each stage

14. ML Approach — why XGBoost (fast, interpretable, works on tabular data) and why LSTM (designed for time-series sequences), each explained in one paragraph with no jargon
15. Expected Output — a hand-drawn or PowerPoint mockup of the Streamlit app showing risk score, trend indicator, and SHAP chart
16. Timeline — a 20-week table showing Phase 1, Phase 2, and Phase 3 milestones

Phase 2 — Partial Prototype (Weeks 4–9)

Deliverable: Working Jupyter Notebook with trained XGBoost model, SHAP explanations, and saved .pkl files

Week	Task	Output
Week 4	Run full data pipeline in Colab: load labevents.csv, filter to 8 kidney item IDs, map names, merge with ICD labels from diagnoses_icd.csv, aggregate per admission, pivot to wide format. Save intermediate result.	ckd_training_ready.csv — clean wide-format training table with 25 feature columns and ckd_label
Week 5	Compute creatinine slope per patient using groupby + np.polyfit. Merge slope into training table. Verify label distribution with value_counts(). Apply SMOTE. Confirm both classes are balanced.	Training table with 25 features + slope + ckd_label. Balanced class counts confirmed.
Week 6	Apply MinMaxScaler (fit on X_train only). Perform 80/20 train/test split. Train XGBoost with specified hyperparameters. Print classification_report on test set. Plot and save confusion matrix.	XGBoost trained. AUC, F1, precision, recall documented.
Week 7	Run 5-fold cross-validation. Print mean AUC and std. Run ablation: train same model WITHOUT creat_slope, compare AUC to show slope improves performance. Document both numbers.	Cross-validation results. Ablation comparison table showing slope contribution.
Week 8	Generate SHAP global_summary_plot (top 10 features). Generate SHAP waterfall_plot for one high-risk patient from the test set. Save both as PNG files.	Two SHAP visualisations. Interpretation written in notebook markdown.
Week 9	Save ckd_model_xgb.pkl, scaler.pkl, feature_cols.pkl with joblib. Build a minimal 30-line Streamlit form: load model, show 8 number inputs, output risk % on button click. Confirm it runs locally.	Phase 2 deliverable: Colab notebook + 3 model files + working minimal app.py

Phase 3 — Full System (Weeks 10–20)

Deliverable: Complete application with LSTM, full Streamlit UI, deployment, and research paper draft

Weeks	Task	Output
10–11	Build patient sequence arrays from the admission feature table. Group by subject_id, sort by admission date, take 3 consecutive admissions, extract mean values of 8 tests per admission. Save X_sequences.npy and y_labels.npy.	LSTM-ready arrays. X_sequences.npy shape: (N_patients, 3, 8)
12–13	Build LSTM in TensorFlow/Keras with the specified architecture. Train with EarlyStopping. Plot training and validation loss curves. Evaluate: AUC, F1, recall, confusion matrix.	ckd_model_lstm.keras. Training loss plot. LSTM evaluation metrics documented.
13–14	Build the XGBoost vs LSTM comparison table. Run ablation experiment: XGBoost without slope vs XGBoost with slope. Fill in all metric values. This becomes Table 1 in the research paper.	Model comparison table with all metric values filled in — core result for paper
14–15	Implement PDF extraction function using PyMuPDF for computer-generated PDFs. Use regex patterns to extract creatinine, BUN, potassium, sodium, hemoglobin values by name. Test on 2 real blood report PDFs.	PDF extractor function working. Tested on real reports. Values extracted correctly.
15–16	Optionally add LLM explanation layer: after risk score, call Claude API (console.anthropic.com, free tier) to generate a 2-3 sentence clinical interpretation. Pass risk score, top SHAP features, and patient values in the prompt.	AI summary section in Streamlit. Works with API key configured.
16–17	Build the complete app.py: multi-visit input (slider for 2-5 visits), 8 fields per visit, compute features on button click, show XGBoost + LSTM scores side by side, creatinine trend indicator, SHAP waterfall, optional LLM summary, disclaimer.	Full app.py — complete and functional locally
17–18	End-to-end testing with the 3 demo scenarios from Section 12. Confirm correct colour coding, trend direction, SHAP direction, and model agreement. Document results.	Test report confirming correct outputs for all 3 scenarios
18–19	Deploy on Streamlit Community Cloud: push project to GitHub (include models/ folder), connect repo at share.streamlit.io, deploy. Confirm app runs at public URL.	Live public URL of deployed app
19–20	Write research paper draft in IEEE two-column format using Overleaf (free). Complete all sections: Abstract, Introduction, Related Work, Methodology, Experiments, Results, Discussion, Conclusion. Minimum 6 pages.	Paper draft ready for faculty review

7. Application User Flow

7.1 Step-by-Step User Journey

17. Doctor opens the web app in a browser (local at localhost:8501 or deployed URL).
18. App title 'Temporal Blood Biomarker Analysis — Kidney Disease Risk Detector' is shown. Research disclaimer is displayed below the title.
19. User selects number of available past blood reports using a slider (2 to 5 visits).
20. For each visit, a labelled input block appears with 8 number_input fields: Creatinine, BUN, Potassium, Sodium, Hemoglobin, Bicarbonate, Calcium, Glucose. Each field shows units and normal range as help text.
21. User fills in values from oldest visit (Visit 1) to most recent visit (Visit N), in chronological order.
22. User clicks the 'Analyse Reports' button.
23. App computes: mean_val and max_val for each test across entered visits, ever_abnormal based on whether value exceeds the clinical threshold, and creat_slope using np.polyfit on the creatinine values.
24. XGBoost model predicts CKD risk probability. Displayed as risk percentage with colour-coded card.
25. Creatinine trend displayed with directional indicator and plain-English caption: 'Creatinine is rising (slope +0.35 per visit) — factored into the risk score.'
26. LSTM model predicts in parallel. Both scores displayed side by side in two metric cards.
27. SHAP waterfall plot rendered below the scores, showing the top 10 features and their contribution magnitude and direction.
28. Optional LLM summary (if API key is configured): 2-3 sentence clinical interpretation generated and displayed in an info box.
29. User can adjust values and re-analyse without reloading the page.

7.2 Input Bounds and Validation

Field	Min	Max	Units	Action If Out of Range
Creatinine	0.1	20.0	mg/dL	st.warning() shown: 'Value outside expected range'
BUN	1.0	150.0	mg/dL	st.warning() shown
Potassium	2.0	8.0	mEq/L	st.warning() shown
Sodium	100.0	160.0	mEq/L	st.warning() shown
Hemoglobin	3.0	20.0	g/dL	st.warning() shown
Bicarbonate	10.0	40.0	mEq/L	st.warning() shown
Calcium	5.0	15.0	mg/dL	st.warning() shown
Glucose	50.0	500.0	mg/dL	st.warning() shown

8. Non-Functional Requirements

Category	Requirement
Performance	XGBoost prediction must complete in under 2 seconds on a standard laptop. LSTM inference under 5 seconds. SHAP computation under 10 seconds.
Usability	All input fields must show placeholder text with example values. Risk result must be visible without scrolling on a 1366x768 screen or larger.
Reliability	App must handle any NaN values in user input gracefully by filling with training-set median rather than crashing.
Privacy	No patient data entered by the user is stored, logged, sent to any server, or persisted between sessions. All computation is in memory only.
Reproducibility	random_state=42 set for SMOTE, train_test_split, XGBoost, and LSTM weight initialisation. Results must be reproducible across separate runs on the same machine.
Portability	App must run on any laptop with Python 3.8 or newer and the listed libraries. Must also run on Streamlit Community Cloud without modification.
Transparency	Disclaimer displayed at top of app: 'This tool is for research and educational purposes only. It does not diagnose, treat, or replace medical advice from a qualified physician.'
File size	All saved model files combined must be under 100 MB for GitHub compatibility. XGBoost models are typically under 5 MB; LSTM models under 20 MB.
Maintainability	Code must be split into clearly commented sections: data loading, feature engineering, model training, model evaluation, app. Each section runnable independently.

9. Research Paper Outline

9.1 Target Venues

Venue	Type	Open Access	URL
IEEE Access	Journal	Yes (free to read)	ieeaccess.ieee.org
MDPI Applied Sciences	Journal	Yes	mdpi.com/journal/applsci
Springer Journal of Medical Sys.	Journal	Hybrid	springer.com/journal/10916
IEEE CBMS	Conference	Published proceedings	cbms-conf.org
ICMLA	Conference	Published proceedings	icmla-conference.org

9.2 Paper Section Breakdown

Section	Content	Target Length
Abstract	Problem statement (CKD, snapshot gap), proposed solution (temporal features, MIMIC-IV, XGBoost + LSTM, SHAP), key result (AUC score, improvement over baseline)	150 words
Introduction	CKD statistics, why early detection matters, why static ML approaches fail, four specific novelty claims this paper makes	1 page
Related Work	5-7 papers: UCI CKD dataset papers (what they did, accuracy, limitation), MIMIC-based disease prediction papers, SHAP in clinical ML, longitudinal EHR modeling	1 to 1.5 pages
Dataset & Preprocessing	MIMIC-IV demo description, labevents.csv column breakdown, ICD-10 label derivation, 8-feature selection rationale, missing value strategy	0.5 page
Feature Engineering	Per-admission aggregation formulas, creatinine slope formula and example, LSTM sequence construction with input shape specification	0.5 to 1 page
Methodology	XGBoost architecture with hyperparameter justification, LSTM architecture diagram, SMOTE rationale, evaluation protocol (5-fold CV + held-out test)	1 page
Results	Table 1: Ablation (with slope vs without). Table 2: XGBoost vs LSTM full comparison. Figure 1:	1.5 pages

Section	Content	Target Length
	SHAP summary plot. Figure 2: SHAP waterfall for one patient. Figure 3: LSTM loss curves.	
Discussion	What SHAP revealed (which features matter most), why slope improves AUC, clinical interpretation of results, limitations (100-patient demo, no external validation)	0.5 page
Conclusion	Summary of four contributions, future work: full MIMIC-IV credential access, external dataset validation, real-time clinical integration	0.25 page

9.3 Key Experiments to Conduct and Report

Experiment	What to Measure	Where It Appears in Paper
XGBoost without creat_slope feature	AUC, F1, Recall on test set	Table 1 — Ablation (Baseline)
XGBoost with creat_slope feature	AUC, F1, Recall on test set — compare to row above	Table 1 — Ablation (Proposed)
XGBoost 5-fold cross-validation	Mean AUC +/- std	Table 2 — Main Comparison
LSTM 5-fold cross-validation	Mean AUC +/- std — compare to XGBoost	Table 2 — Main Comparison
SHAP global importance	Top 5 features by mean absolute SHAP value across test patients	Figure 1 — SHAP Summary Plot
SHAP local explanation	Waterfall for one high-risk patient from Scenario C (Section 12)	Figure 2 — Per-Patient Explanation
Comparison to prior UCI-based papers	This project AUC vs AUC reported in 3 published papers on UCI CKD	Table 3 — Comparison with Prior Work

10. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
MIMIC-IV demo has only ~100 patients — model may overfit or have high variance	High	High	Use 5-fold cross-validation. Apply SMOTE. Report std alongside mean AUC. Supplement with UCI CKD dataset for additional XGBoost training data if needed.
Not enough patients have 3+ admissions for LSTM training sequences	Medium	Medium	Relax minimum to 2 visits. Augment by treating each consecutive pair as a sequence. Be transparent about augmentation in the paper — this is acceptable methodology.
diagnoses_icd.csv yields too few CKD-positive patients for training	Medium	High	Apply KDIGO clinical rule fallback (Section 2.4). Document clearly in paper. Supplementing with UCI dataset labels is also acceptable for training.
AUC does not reach 0.80 on test set	Medium	Medium	Try: adding ever_abnormal flags as additional features, tuning max_depth and n_estimators via GridSearchCV, or combining UCI and MIMIC data for training.
app.py crashes with KeyError due to column order mismatch	High	Low	Always load feature_cols.pkl and reorder the input DataFrame with input_df = input_df[feature_cols] before calling scaler.transform(). This is the most common deployment bug.
LSTM consistently underperforms XGBoost on this small dataset	High	Low	This is expected — report it honestly. It is a valid research finding: 'XGBoost with temporal slope features matches LSTM on small datasets, while being faster and more interpretable.' This finding itself is publishable.
Streamlit Community Cloud fails to deploy due to file size	Low	Low	Ensure combined model files are under 100 MB. If LSTM is too large, exclude it from the cloud deployment and demonstrate it locally.

11. Common Mistakes and How to Avoid Them

Mistake	Consequence	Correct Approach
Fitting the scaler on the full dataset before the train/test split	Data leakage — test performance is inflated and meaningless; the scaler has 'seen' test values	Always split first. Then: <code>scaler.fit(X_train)</code> , <code>scaler.transform(X_train)</code> , <code>scaler.transform(X_test)</code> . Never <code>scaler.fit(X_all)</code> .
Reporting only accuracy as the model quality metric	A model that predicts 'CKD = 1' for every single patient will have 67% accuracy but will have zero precision and will never be useful	Always report AUC, F1, recall, and precision. State in the paper that accuracy is not used due to potential class imbalance.
Skipping SMOTE when classes are imbalanced	Model learns to predict the majority class and ignores the minority — high accuracy, zero recall on CKD patients	Check <code>value_counts()</code> after creating labels. If ratio exceeds 60:40, apply SMOTE before training. Always apply SMOTE to training data only, not test data.
Not saving <code>feature_cols.pkl</code>	In <code>app.py</code> , the DataFrame column order may differ from training, causing the model to assign values to wrong features silently — predictions are wrong without any error	Always: <code>joblib.dump(feature_cols, 'feature_cols.pkl')</code> . In <code>app.py</code> : load it and apply <code>input_df = input_df[feature_cols]</code> before scaling.
Treating LSTM as definitely better than XGBoost	LSTM needs large sequential datasets to outperform. On 80 training sequences (typical for MIMIC demo), XGBoost will likely win. If you expect LSTM to win, you will be confused by results.	Run both, compare honestly, report actual numbers. XGBoost winning is a valid and publishable finding.
Using only the SHAP global summary plot and skipping the per-patient waterfall	The demo is not impressive — the examiner cannot see the AI reasoning for any individual patient	Always include both: <code>summary_plot</code> in the paper (Figure 1), <code>waterfall_plot</code> live in the Streamlit demo for Scenario C (the high-risk patient)
Training XGBoost on the full dataset (including test set)	Perfect accuracy on test set because the model memorised those examples during training; reported result is completely fabricated	Use <code>train_test_split</code> before training. The model sees <code>X_train</code> only. <code>X_test</code> is held out and used only for the final evaluation — never for training.

12. Demo Scenarios for Phase 3 Evaluation

Use these three test scenarios during the live demo. They are designed to produce clearly distinct outputs that illustrate the temporal dimension of the system.

Scenario A — Healthy Patient (Expected: Green / Low Risk)

Visit	Creatinine	BUN	Potassium	Sodium	Hemoglobin	Bicarbonate	Calcium	Glucose
Visit 1 (3 months ago)	0.9	12	4.0	138	14.2	25	9.5	88
Visit 2 (2 months ago)	0.9	13	4.1	139	14.0	25	9.5	90
Visit 3 (last month)	0.8	12	3.9	138	14.3	24	9.6	87

Expected: Low risk score. Creatinine slope is negative (falling). All values within normal range. SHAP should show no features with large positive contribution.

Scenario B — Moderate Risk Patient (Expected: Amber / Moderate Risk)

Visit	Creatinine	BUN	Potassium	Sodium	Hemoglobin	Bicarbonate	Calcium	Glucose
Visit 1 (3 months ago)	1.3	22	4.5	135	12.5	22	9.0	145
Visit 2 (2 months ago)	1.4	25	4.6	134	12.2	21	8.9	152
Visit 3 (last month)	1.5	28	4.7	133	12.0	21	8.8	160

Expected: Moderate risk. Rising creatinine (slope +0.1). Mildly elevated BUN. Hemoglobin declining. Glucose rising — indicates diabetic stress on kidneys.

Scenario C — High Risk Patient (Expected: Red / High Risk)

Visit	Creatinine	BUN	Potassium	Sodium	Hemoglobin	Bicarbonate	Calcium	Glucose
Visit 1 (6 months ago)	1.5	30	4.8	131	11.0	19	8.5	220
Visit 2 (3 months ago)	1.9	40	5.1	129	10.5	18	8.2	245

Visit	Creatinine	BUN	Potassium	Sodium	Hemoglobin	Bicarbonate	Calcium	Glucose
Visit 3 (last month)	2.4	52	5.4	127	10.0	17	7.9	270

Expected: High risk. Creatinine slope +0.45 (fastest rise). BUN severely elevated. Potassium above normal. Hemoglobin declining. Bicarbonate low (acidosis). SHAP should highlight creat_slope and max_Creatinine as the top contributors.

Demo Talking Points

- Scenarios A and C have the same number of visits. Different trajectories produce completely different risk scores — the slope is the signal.
- Point at the SHAP waterfall for Scenario C: 'This is why XGBoost gave this score. Not just because creatinine is high today — but because it has been rising at 0.45 mg/dL per visit for six months.'
- Compare the XGBoost and LSTM scores for Scenario C. If both flag high risk, say: 'Both models agree — this increases confidence in the result.'
- If asked about accuracy: 'We measured a mean AUC of X using 5-fold cross-validation on the MIMIC-IV demo dataset. Our temporal approach with the creatinine slope feature improves AUC by Y% compared to the same model without temporal information — as shown in our ablation study.'

13. Project File Structure

Your project folder on your laptop and on GitHub must follow this structure:

Path	Purpose
ckd_project/	Root project folder — this is your GitHub repository
— data/	Raw and processed data files (add to .gitignore if files are large)
— labevents.csv	Raw MIMIC-IV demo lab events from PhysioNet
— diagnoses_icd.csv	ICD-10 diagnosis codes from PhysioNet
— ckd_training_ready.csv	Processed wide-format XGBoost training table (output of Phase 2)
— X_sequences.npy	LSTM input sequences array — shape (N, 3, 8)
— y_labels.npy	LSTM label array
— notebooks/	All Jupyter notebooks — one per pipeline stage
— 01_data_exploration.ipynb	EDA on raw labevents.csv — understand structure and identify item IDs
— 02_feature_engineering.ipynb	Aggregation, pivoting, slope computation, LSTM sequence construction
— 03_xgboost_training.ipynb	XGBoost: SMOTE, scaling, training, 5-fold CV, SHAP, ablation
— 04_lstm_training.ipynb	LSTM: sequence scaling, model build, training, loss curves, evaluation
— models/	All saved model files — commit these to GitHub (they are small)
— ckd_model_xgb.pkl	Trained XGBoost model
— scaler.pkl	MinMaxScaler fitted on XGBoost training data
— feature_cols.pkl	Ordered list of feature column names — critical for app.py
— ckd_model_lstm.keras	Trained LSTM model
— app.py	Streamlit web application — run with: streamlit run app.py
— requirements.txt	All Python library dependencies — used by Streamlit Cloud
— .gitignore	Exclude data/ folder (large files) and __pycache__
— README.md	Project description, setup instructions, dataset URL, and model performance summary

